week3_inside an FCS file

Macarena López-Mayorga

2026-03-05

We are going to work with some fcs files located in the week3. First, we have to locate them and name them

```
PathToDataFolder <- file.path("C:/Users/Macarena/CytometryInR/course/03_InsideFCSFile/data")
PathToDataFolder
```

[1] "C:/Users/Macarena/CytometryInR/course/03_InsideFCSFile/data"

```
fcs_files <- list.files(PathToDataFolder, full.names=TRUE, pattern=".fcs")
fcs_files
```

[1] "C:/Users/Macarena/CytometryInR/course/03_InsideFCSFile/data/CellCounts4L_AB_05_ND050_05.fcs

Next, we are going to use flowCore. We have to call it using library function

```
library(flowCore)
```

we are going to use the read.fcs function. Let's see the help documentation

```
#|eval: FALSE
?read.FCS
```

```
No documentation for 'read.FCS' in specified packages and libraries:
you could try '??read.FCS'
```

```
firstfile <- fcs_files[1]
firstfile
```

For the function read.FCS, let's use just the first argument for the moment

```
read.FCS(filename=firstfile)
```

flowFrame object 'CellCounts4L_AB_05-ND050-05.fcs' with 100 cells and 61 observables: name desc range minRange maxRange $P1TimeNA2721400272139P2$ UV1-A NA 4194304 -111 4194303 $P3UV2 - ANA4194304 - 1114194303P4$ UV3-A NA 4194304 -111 4194303 $P5UV4 - ANA4194304 - 1114194303$.....P57 R4-A NA 4194304 -111 4194303 $P58R5 - ANA4194304 - 1114194303P59$ R6-A NA 4194304 -111 4194303 $P60R7 - ANA4194304 - 1114194303P61$ R8-A NA 4194304 -111 4194303 599 keywords are stored in the 'description' slot

R gives back what resembles a table, with the number of cells, the name of files, the name of detectors (column labelled as "name")

```
flowFrame <- read.FCS(filename=firstfile, transformation = FALSE, truncate_max_range = FALSE)
```

Let's see what kind of file is now

```
class(flowFrame)
```

[1] "flowFrame" attr(,"package") [1] "flowCore" This is a new type of file named as "S4 object". At the right panel, there is a drop-down in the flowFrame file with 3 slots. The first one, "exprs", has a grid symbol. If we click it, we can see a table with the individual detectors and the data of each cell (until 100). To study this matrix in more detail, we are going to assign a value. Because this is a S4 object, we have to indicate the value specifically to this slot.

exprs slot

```
MFI_Matrix <- flowFrame@exprs
```

Alternatively, we can use the Bioconductor helper function

```
MFI_Matrix_Alternate <- exprs(flowFrame)
```

With this last code, we can see a few rows, since all the data will be very complicated to work with

```
head(MFI_Matrix, 5)
```

```
   Time       UV1-A        UV2-A      UV3-A       UV4-A       UV5-A       UV6-
A       UV7-A      UV8-A        UV9-A      UV10-A     UV11-A     UV12-A
```

[1,] 38823 37.79983 -184.479996 353.87714 1106.22998 1145.18140 2130.21899 4376.34277 3246.7952 32050.65039 8123.2637 1992.5785 1070.3323 [2,] 39780 234.23021 -98.456429 26.43876 70.65833 -89.93541 -29.14263 -26.34649 -162.6675 17.98848 271.9152 154.8575 163.5411 [3,] 267292 -117.96355 40.732426 473.94574 1177.86975 1516.70935 1985.46130 3658.87671 4140.1724 59792.16406 14013.7969 3427.4324 1668.7588 [4,] 128101 289.87671 -2.723389 -54.11960 -163.71489 32.62989 -134.90411 739.71960 402.9025 427.37534 315.6364 -223.0423 145.7121 [5,] 255221 -104.50541 -71.163338 567.57562 610.23627 1416.61328 2868.16040 4034.58789 3234.6626 40126.46484 10325.0371 1974.0907 1033.8450 UV13-A UV14-A UV15-A UV16-A SSC-H SSC-A V1-A V2-A V3-A V4-A V5-A V6-A V7-A V8-A [1,] 956.43573 290.8685 385.49921 670.97687 657613 750760.12 1171.1390 154.5628 1346.4488525 1706.9260 1923.50940 898.2527 3162.55371 83596.5078 [2,] -32.81524 -104.9198 103.41382 71.41528 83481 81552.85 266.2424 705.2527 244.3218689 341.0508 381.15939 87.1600 151.68785 119.5544 [3,] 1071.14636 730.1430 214.93053 252.75406 890845 1183519.00 1196.0931 1183.1105 2087.0092773 824.6352 1635.27258 1613.9069 4653.16260 176981.6094 [4,] 127.03777 -207.8978 -55.37944 -45.10131 75103 72457.33 227.9926 556.9189 -0.8137281 205.4732 18.12125 179.6371 -69.50061 -132.4348 [5,] -21.57245 273.6271 960.16290 341.20633 415791 501690.97 717.2498 929.9780 1358.4512939 788.6506 1208.81006 1156.7040 3118.42627 104951.2578 V9-A V10-A V11-A V12-A V13-A V14-A V15-A V16-A FSC-H FSC-A SSC-B-H SSC-B-A B1-A [1,] 32506.7617 27161.5137 6236.072754 2220.303223 2023.39966 753.589355 510.9540 228.34962 1055905 1217097.50 716733 815959.06 606.6683 [2,] -79.7691 -109.1783 -73.991196 114.375542 -11.53453 124.986206 -207.3494 -28.96272 79696 83439.11 104575 103132.83 195.2795 [3,] 69236.4297 57626.8984 13175.838867 4534.874023 3434.38989 1995.172363 1321.8030 615.05560 1092481 1453969.38 757351 982038.31 2010.5110 [4,] 129.2739 231.8918 -5.473321 8.792875 -24.62049 -8.212234 -133.7503 -34.32619 64760 60415.23 67955 66806.13 -146.8936 [5,] 42090.0781 34104.3164 7620.552734 3103.544189 2426.43359 650.836304 290.2892 473.32599 1038362 1184479.00 425296 522873.12 1015.5981 B2-A B3-A B4-A B5-A B6-A B7-A B8-A B9-A B10-A B11-A B12-A B13-A [1,] 416.98294 4172.4712 192400.0938 93929.9375 54236.3320 19342.6445 10507.24219 9498.1270 4465.50928 1668.048096 2199.9475 1581.7345 [2,] 333.25662 332.1675 -230.9639 196.7810 292.8945 -187.2845 70.90886 -334.3563 188.05545 -663.359619 -163.2331 27.9856 [3,] 2150.21826 10106.9551 437801.5625 212176.1562 124294.3594 45068.3008 24289.59180 22500.1914 10624.99219 4684.497559 3471.7749 2727.3904 [4,] -34.90987 165.7988 675.0156 136.3076 482.8665 133.0948 432.28091 246.8794 -94.44906 2.905877 106.8489 283.3633 [5,] 639.77527 6034.0200 244022.6094 118871.4609 68616.9688 24067.7949 14182.12793 13019.7100 5577.33984 2753.355957 1279.3009 1423.0276 B14-A R1-A R2-A R3-A R4-A R5-A R6-A R7-A R8-A [1,] 1487.56860 147.1335 129.867630 35.90353 267.23999 49.79849 -732.7097 42.83144 248.56728 [2,] 205.82298 -142.9224 66.516052 113.63218 -94.41375 98.13978 143.4497 -263.28741 -85.83299 [3,] 2371.95850 -128.3749 -105.482544 726.48547 18.87000 95.47879 -194.4526 -84.08820 -301.46066 [4,] 33.24665 127.5455 122.607941 37.83584 -82.87500 -343.83768 82.3745 60.27896 -94.38461 [5,] 1565.72742 -266.4482 -3.350622 -178.39566 -117.10875 -100.10384 -182.0066 184.36417 186.17207

```
ColumnNames <- colnames(MFI_Matrix)
ColumnNames
```

$P1N $P2N $P3N $P4N $P5N $P6N $P7N $P8N $P9N $P10N $P11N $P12N $P13N $P14N $P15N “Time” “UV1-A” “UV2-A” “UV3-A” “UV4-A” “UV5-A” “UV6-A” “UV7-A” “UV8-A” “UV9-A” “UV10-A” “UV11-A” “UV12-A” “UV13-A” “UV14-A” $P16N $P17N $P18N $P19N $P20N $P21N $P22N $P23N $P24N $P25N $P26N $P27N $P28N $P29N $P30N “UV15-A” “UV16-A” “SSC-H” “SSC-A” “V1-A” “V2-A” “V3-A” “V4-A” “V5-A” “V6-A” “V7-A” “V8-A” “V9-A” “V10-A” “V11-A” $P31N $P32N $P33N $P34N $P35N $P36N $P37N $P38N $P39N $P40N $P41N $P42N $P43N $P44N $P45N “V12-A” “V13-A” “V14-A” “V15-A” “V16-A” “FSC-H” “FSC-A” “SSC-B-H” “SSC-B-A” “B1-A” “B2-A” “B3-A” “B4-A” “B5-A” “B6-A” $P46N $P47N $P48N $P49N $P50N $P51N $P52N $P53N $P54N $P55N $P56N $P57N $P58N $P59N $P60N “B7-A” “B8-A” “B9-A” “B10-A” “B11-A” “B12-A” “B13-A” “B14-A” “R1-A” “R2-A” “R3-A” “R4-A” “R5-A” “R6-A” “R7-A” $P61N “R8-A”

Above of each receptor, appears the pattern “$P#N”, that remind us to the pattern that appears when we ran read.FCS (see above)

::: {.cell}

```
str(ColumnNames)
```

:::

Named chr [1:61] “Time” “UV1-A” “UV2-A” “UV3-A” “UV4-A” “UV5-A” “UV6-A” “UV7-A” “UV8-A” “UV9-A” “UV10-A” “UV11-A” “UV12-A” “UV13-A” “UV14-A” … - attr(*, “names”)= chr [1:61] “$P1N""P2N” “$P3N""P4N” …

Here, we see that, each value has a corresponding index name as well. Let’s see using class function

::: {.cell}

```
class(ColumnNames)
```

:::

[1] “character”

we just see that all is considered as character, but it doesn’t inform us about the index. That’s why it is best to use several codes, to obtain as much information as possible. We can also remove the names and left just the values:

::: {.cell}

```
unname(ColumnNames)
```

:::

[1] “Time” “UV1-A” “UV2-A” “UV3-A” “UV4-A” “UV5-A” “UV6-A” “UV7-A” “UV8-A” “UV9-A” “UV10-A” “UV11-A” “UV12-A” “UV13-A” [15] “UV14-A” “UV15-A” “UV16-A” “SSC-H” “SSC-A” “V1-A” “V2-A” “V3-A” “V4-A” “V5-A” “V6-A” “V7-A” “V8-A” “V9-A” [29] “V10-A” “V11-A” “V12-A” “V13-A” “V14-A” “V15-A” “V16-A” “FSC-H” “FSC-A” “SSC-B-H” “SSC-B-A” “B1-A” “B2-A” “B3-A” [43] “B4-A” “B5-A” “B6-A” “B7-A” “B8-A” “B9-A” “B10-A” “B11-A” “B12-A” “B13-A” “B14-A” “R1-A” “R2-A” “R3-A” [57] “R4-A” “R5-A” “R6-A” “R7-A” “R8-A”

If we look again to the rigth bar, in the drop-down of flowFrame and click in the first slot (exprs) there is another drop-down less user-friendly than the little grid. The columns are represented by []. Inside, in front of the number, it should be a comma (,)

```
MFI_Matrix[,1]
```

[1] 38823 39780 267292 128101 255221 79210 196643 83855 109315 26128 114423 120001 71831 70551 197021 239994 252611 223012 152780 171822 [21] 172611 168464 191503 253015 73885 82221 176641 128533 4117 191632 191229 58093 141776 265894 55593 227555 233212 248578 95165 171934 [41] 1360 251847 195764 147503 118723 1060 90033 253553 268268 74610 23531 150119 226391 201568 179264 79944 196686 252667 117309 3903 [61] 77690 195142 229873 254472 179943 236618 68193 87154 28541 78622 155664 50115 40866 70753 260118 12033 96149 20740 37461 73998 [81] 231939 192329 88649 197664 86006 142486 159539 251298 104864 164090 102380 218968 145182 239323 261272 118979 17202 194277 229284 258723

::: {.cell}

```
MFI_Matrix[,61]
```

:::

[1] 248.567276 -85.832993 -301.460663 -94.384613 186.172073 -461.407745 843.507080 277.516113 -106.166855 281.633545 195.927261 818.865723 [13] 734.996460 209.356476 206.442596 279.859894 518.165222 56.947498 285.751007 857.126343 -94.384613 -213.030518 62.585236 138.409653 [25] 118.012444 328.255768 -61.635056 185.285233 464.384979 5.637739 -66.385956 31.229273 1198.241211 185.475266 873.279419 457.607025 [37] -73.353951 37.880539 729.168640 221.772171 -169.512238 348.272888 -338.391022 845.534119 -4.434176 620.024597 610.269409 -193.900208 [49] 230.830566 -23.754517 607.102112 14.949510 -34.333195 -169.132172 -96.158287 220.631958 125.297165 -15.202891 -126.057304 193.393448 [61] 90.203819 -277.706146 590.505615 911.096619 -92.230873 347.259369 135.559113 369.430267

-62.015125 -180.597672 -146.517868 810.440796 [73] 134.038818 -165.268097 727.711731 -88.746880 62.901962 203.275330 436.196289 -242.676147 -40.857769 222.278946 -170.272385 525.513245 [85] -41.491222 176.670258 201.501648 175.530045 329.839386 474.140167 -48.142490 -174.833252 46.052090 357.584656 -26.541714 191.493088 [97] 211.320190 124.790398 -113.324883 343.268616

If we use a column number that doesn´t exist:

::: {.cell}

```
MFI_Matrix[,350]
```

:::

Error in `MFI_Matrix[, 350]`: ! subscript out of bounds

It gives us an error message.

So, we know how represent the columns [] with a comma inside before a specific number. For rows, it is also used [] but the order is altered: first, we put a specific number and then, a comma.

```
MFI_Matrix[1,]
```

```
    Time          UV1-A          UV2-A          UV3-A          UV4-A          UV5-
A          UV6-A          UV7-A          UV8-A          UV9-A
```

38823.00000 37.79983 -184.48000 353.87714 1106.22998 1145.18140 2130.21899 4376.34277 3246.79517 32050.65039 UV10-A UV11-A UV12-A UV13-A UV14-A UV15-A UV16-A SSC-H SSC-A V1-A 8123.26367 1992.57849 1070.33228 956.43573 290.86853 385.49921 670.97687 657613.00000 750760.12500 1171.13904 V2-A V3-A V4-A V5-A V6-A V7-A V8-A V9-A V10-A V11-A 154.56281 1346.44885 1706.92603 1923.50940 898.25269 3162.55371 83596.50781 32506.76172 27161.51367 6236.07275 V12-A V13-A V14-A V15-A V16-A FSC-H FSC-A SSC-B-H SSC-B-A B1-A 2220.30322 2023.39966 753.58936 510.95404 228.34962 1055905.00000 1217097.50000 716733.00000 815959.06250 606.66833 B2-A B3-A B4-A B5-A B6-A B7-A B8-A B9-A B10-A B11-A 416.98294 4172.47119 192400.09375 93929.93750 54236.33203 19342.64453 10507.24219 9498.12695 4465.50928 1668.04810 B12-A B13-A B14-A R1-A R2-A R3-A R4-A R5-A R6-A R7-A 2199.94751 1581.73450 1487.56860 147.13348 129.86763 35.90353 267.23999 49.79849 -732.70966 42.83144 R8-A 248.56728

We can also ask for a specific value of the table, by putting the index number of the row and
A detector for the first acquired cell (knowing that UV1-A is the 2nd column):

```
MFI_Matrix[1,2]
```

UV1-A 37.79983

parameters slot

This seems to be a more complex slot, with 4 sub-slots in it

varMetadata

It is a table with a column of metadata names, as we can see when we click in the grid. These look reminiscent of what we saw at the top of the read.FCS() column outputs previously (see above)

data

If we click on the grid, we will see the actual content that was displayed. Let's try to retrieve the data contained within this slot and save it as it's own variable/object within our R session. First, we need to open flowFrame object, then use @ to get inside its parameters slot. Since parameters is also a complex object (AnnotatedDataFrame specifically), we will need to use another @ to get inside its data slot:

```
ParameterData <- flowFrame@parameters@data
head(ParameterData, 10)
```

```
  name desc   range   minRange maxRange
```

$P1Time < NA >$ 2721400.00000272139P2 UV1-A 4194304 -111.00000 4194303 $P3UV2 - A <$ $NA >$ 4194304 – 111.000004194303P4 UV3-A 4194304 -111.00000 4194303 $P5UV4 - A <$ $NA >$ 4194304 – 111.000004194303P6 UV5-A 4194304 -111.00000 4194303 $P7UV6 - A <$ $NA >$ 4194304 – 111.000004194303P8 UV7-A 4194304 -26.34649 4194303 $P9UV8 - A <$ $NA >$ 4194304 – 111.000004194303P10 UV9-A 4194304 0.00000 4194303

We can do the same using the Bioconductor helper function

```
ParameterData_Alternate <- parameters(flowFrame)@data
head(ParameterData_Alternate, 10)
```

```
  name desc   range   minRange maxRange
```

$P1Time < NA >$ 2721400.00000272139P2 UV1-A 4194304 -111.00000 4194303 $P3UV2 - A <$ $NA >$ 4194304 − 111.000004194303P4 UV3-A 4194304 -111.00000 4194303 $P5UV4 - A <$ $NA >$ 4194304 − 111.000004194303P6 UV5-A 4194304 -111.00000 4194303 $P7UV6 - A <$ $NA >$ 4194304 − 111.000004194303P8 UV7-A 4194304 -26.34649 4194303 $P9UV8 - A <$ $NA >$ 4194304 − 111.000004194303P10 UV9-A 4194304 0.00000 4194303

Now, we are going to run the str function

```
str(ParameterData)
```

'data.frame': 61 obs. of 5 variables: $ name : 'AsIs' Named chr "Time" "UV1-A" "UV2-A" "UV3-A" … ..- attr(, *"names")= chr [1:61] "P1N""P2N" "P3N""P4N" … $ desc : 'AsIs' Named chr NA NA NA NA … ..- attr(*, "names")= chr [1:61] NA NA NA NA … $ range : num 272140 4194304 4194304 4194304 4194304 … $ minRange: num 0 -111 -111 -111 -111 … $ maxRange: num 272139 4194303 4194303 4194303 4194303 …

We can see this class of object is a "data.frame". This is one of the more common object types in R. Each of the columns appears to be designated by a $ followed by the column name, and then type of column (numeric, character, etc).

If we are trying to see these columns in R, we notice that data.frame is not like the previous S4 class objets we interacted with, as the @ symbol after doesn't bring up any suggestions

::: {.cell}

```
ParameterData@
```

:::

Error: ! Can't parse incomplete input

By contrast, adding the $ we see that R suggests us different column names (name, range, minRange, etc). Similar to what we saw with a matrix, we can subset a data.frame based on the column or row index using square brackets [].

```
ParameterData[,1]
```

$P1N $P2N $P3N $P4N $P5N $P6N $P7N $P8N $P9N $P10N $P11N $P12N $P13N "Time" "UV1-A" "UV2-A" "UV3-A" "UV4-A" "UV5-A" "UV6-A" "UV7-A" "UV8-A" "UV9-A" "UV10-A" "UV11-A" "UV12-A" $P14N $P15N $P16N $P17N $P18N $P19N $P20N $P21N $P22N $P23N $P24N $P25N $P26N "UV13-A" "UV14-A" "UV15-A" "UV16-A" "SSC-H" "SSC-A" "V1-A" "V2-A" "V3-A" "V4-A" "V5-A" "V6-A" "V7-A" $P27N $P28N $P29N

$P30N $P31N $P32N $P33N $P34N $P35N $P36N $P37N $P38N $P39N “V8-A” “V9-A” “V10-A” “V11-A” “V12-A” “V13-A” “V14-A” “V15-A” “V16-A” “FSC-H” “FSC-A” “SSC-B-H” “SSC-B-A” $P40N $P41N $P42N $P43N $P44N $P45N $P46N $P47N $P48N $P49N $P50N $P51N $P52N “B1-A” “B2-A” “B3-A” “B4-A” “B5-A” “B6-A” “B7-A” “B8-A” “B9-A” “B10-A” “B11-A” “B12-A” “B13-A” $P53N $P54N $P55N $P56N $P57N $P58N $P59N $P60N $P61N “B14-A” “R1-A” “R2-A” “R3-A” “R4-A” “R5-A” “R6-A” “R7-A” “R8-A”

The individual detectors or fluorophore appear under “name”. For now, based on what we know, the $P# appears to be some sort of name being used as an internal consistent reference to the respective.

dimLabels Let’s see the parameters for dimLabels

::: {.cell}

```
flowFrame@parameters@dimLabels
```

:::

[1] “rowNames” “columnNames”

classVersion

We are going to continue with the last slot

```
flowFrame@parameters@.__classVersion__
```

AnnotatedDataFrame “1.1.0” # *Description* When we drop-down this slot, we see a very long list is opened. To make easier the managment of the list, it could be better to retrieve some of the data via the console. To retrieve the list itself, we would need to access the description slot of the flowFrame object. Since it is a slot, we will need to use the @ accessor.

```
DescriptionList <- flowFrame@description
DescriptionList
```

The returned list is a little too large to reasonably explore. We can attempt to subset using the head() function as shown below:

```
head(DescriptionList, 5)
```

`BEGINANALYSIS` [1] "0"

`BEGINDATA` [1] "33312"

`BEGINSTEXT` [1] "0"

`BTIM` [1] "13:55:29.85"

`BYTEORD` [1] "4,3,2,1" Now, we just see these 5 elements.

We can also use this other code:

```
DescriptionList[1:10]
```

And a Bioconductor helper keyword function:

```
DescriptionList_Alternate <- keyword(flowFrame)
DescriptionList_Alternate
```

If we run the class() function, we can see that DescriptionList is an actual "list".

```
class(DescriptionList)
```

[1] "list" This is in contrast to the vectors we have previously generated. While these are also list like, they are what are known as an atomic list, which contain values that are all either characters, numerics or logicals. For example:

```
Fluorophores <- c("BV421", "FITC", "PE", "APC")
class(Fluorophores)
```

[1] "character"

```
PanelAntibodyCounts <- c(5, 12, 19, 26, 34, 46, 51)
class(PanelAntibodyCounts)
```

[1] "numeric"

```
SpecimenIndexToKeep <- c(TRUE, TRUE, FALSE, TRUE)
class(SpecimenIndexToKeep)
```

[1] “logical”

A list on the other hand is not restricted to contain objects composed entirely of a certain atomic type. For example, I could include the three previous vectors into a list using the list() function.

```
MyListofVectors <- list(Fluorophores, PanelAntibodyCounts, SpecimenIndexToKeep)
str(MyListofVectors)
```

List of 3 $: chr [1:4] “BV421” “FITC” “PE” “APC” $: num [1:7] 5 12 19 26 34 46 51 $: logi [1:4] TRUE TRUE FALSE TRUE

We can see that with the Description/Keyword list we retrieved from our flowFrame shares a somewhat similar format.

```
str(DescriptionList[1:10])
```

List of 10 $ $BEGINANALYSIS : chr “0” $ $BEGINDATA : chr “33312” $ $BEGINSTEXT : chr “0” $ $BTIM : chr “13:55:29.85” $ $BYTEORD : chr “4,3,2,1” $ $CYT : chr “Aurora” $ $CYTOLIB_VERSION: chr “2.22.0” $ $CYTSN : chr “V0333” $ $DATATYPE : chr “F” $ $DATE : chr “04-Aug-2025”

But in this case, there are also names present ($BEGINANALYSIS, $BEGINDATA, etc). What if we had tried to provide names to our List of Vectors? Would the format match? When we assigned a name to each of the vectors (by providing an equal to =), we get the same kind of structure format to what we see in Description.

::: {.cell}

```
MyNamedListofVectors <- list(FluorophoresNamed=Fluorophores, PanelAntibodyCountsNamed=PanelAn

str(MyNamedListofVectors)
```

:::

List of 3 $ FluorophoresNamed : chr [1:4] “BV421” “FITC” “PE” “APC” $ PanelAntibody-CountsNamed: num [1:7] 5 12 19 26 34 46 51 $ SpecimenIndexToKeepNamed: logi [1:4] TRUE TRUE FALSE TRUE

We could then subsequently be able to isolate items from that list using the $ operator (we get several options when we write the $ symbol). Here an example

::: {.cell}

```
MyNamedListofVectors$FluorophoresNamed
```

:::

Alternatively, we could also access by list index position

::: {.cell}

```
MyNamedListofVectors[1]
```

:::

$FluorophoresNamed [1] “BV421” “FITC” “PE” “APC” Remembering back to the original output from read.FCS() we remember that it mentioned 599 keywords being in the description slot, so now we know that this is what was being referenced. Let’s take a birds eye view of the contents of this list (we are not going to look for every single keyword)

```
Subset <- DescriptionList[1:18]
Subset
```

Early Metadata

Within the initial portion, we are getting back metadata keywords related to where and how the particular file was acquired. Keywords of potential interest include: Start time: What time was the .fcs file acquired

```
DescriptionList$`$BTIM`
```

[1] “13:55:29.85”

Cytometer: What type of cytometer was the .fcs file acquired on

```
DescriptionList$`$CYT`
```

[1] “Aurora”

Cytometer serial Number: Manufacturer Serial Number of the Cytometer

```
DescriptionList$`$CYTSN`
```

[1] “V0333”

FCS File Acquisition Date: What was the date of acquisition

```
DescriptionList$`$DATE`
```

[1] “04-Aug-2025”

Acquisition End Time: What time was acquisition stopped

```
DescriptionList$`$ETIM`
```

[1] “13:55:57.02”

File Name: Name of the .fcs file

```
DescriptionList$`$FIL`
```

[1] “CellCounts4L_AB_05-ND050-05.fcs”

Operator: Who acquired the .fcs file

```
DescriptionList$`$OP`
```

[1] “David Rach”

Detector values

Other important group of keywords are related with the individual detectors for at the time of acquisition.

```
Detectors <- DescriptionList[20:384]
Detectors
```

We get back a huge list of data. Here, you can see the meaning of the last letter of those keywords: For example: $P7B. “B” means number of bits reserved for parameter number n.

```
DescriptionList$`$P7B`
```

[1] “32” For $P7E. E means amplification type for parameter n.

```
DescriptionList$`$P7E`
```

[1] “0,0”

For $P7N. N is the short name for parameter n

```
DescriptionList$`$P7N`
```

[1] “UV6-A”

For $P7R. R is the range for parameter number n

```
DescriptionList$`$P7R`
```

[1] “4194304”

For $P7TYPE. TYPE is the detector type for parameter n

```
DescriptionList$`$P7TYPE`
```

[1] “Raw_Fluorescence”

For $P7V. V is the detector voltage for parameter n.

```
DescriptionList$`$P7V`
```

[1] “506” ## *Middle Metadata* Once we are out of the detector keywords, we find the last of the $Metadata associated keywords.

```
Detectors <- DescriptionList[385:398]
Detectors
```

‘PAR‘ [1] “61”

‘PROJ‘ [1] “CellCounts4L_AB_05”

‘SPILLOVER‘ UV1-A UV2-A UV3-A UV4-A UV5-A UV6-A UV7-A UV8-A UV9-A UV10-A UV11-A UV12-A UV13-A UV14-A UV15-A UV16-A V1-A V2-A V3-A V4-A V5-A [1,] 1e+00 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [2,] 1e-06 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [3,] 0e+00 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [4,] 0e+00 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [5,] 0e+00 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [6,] 0e+00 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [7,] 0e+00 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [8,] 0e+00 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [9,] 0e+00 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 [10,] 0e+00 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 [11,] 0e+00 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 [12,] 0e+00 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 [13,] 0e+00 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 [14,] 0e+00 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 [15,] 0e+00 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 [16,] 0e+00 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 [17,] 0e+00 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 [18,] 0e+00 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 V6-A V7-A V8-A V9-A V10-A V11-A V12-A V13-A V14-A V15-A V16-A B1-A B2-A B3-A B4-A B5-A B6-A B7-A B8-A B9-A B10-A B11-A B12-A B13-A [1,]

0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [2,] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
[3,] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [4,] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 [5,] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [6,] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 [7,] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [8,] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 [9,] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [10,] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 [11,] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [12,] 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 [13,] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [14,] 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [15,] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [16,] 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [17,] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [18,] 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 B14-A R1-A R2-A R3-A R4-A R5-A R6-A R7-A
R8-A [1,] 0 0 0 0 0 0 0 0 0 [2,] 0 0 0 0 0 0 0 0 0 [3,] 0 0 0 0 0 0 0 0 0 [4,] 0 0 0 0 0 0 0 0 0 [5,] 0
0 0 0 0 0 0 0 0 [6,] 0 0 0 0 0 0 0 0 0 [7,] 0 0 0 0 0 0 0 0 0 [8,] 0 0 0 0 0 0 0 0 0 [9,] 0 0 0 0 0 0 0
0 0 [10,] 0 0 0 0 0 0 0 0 0 [11,] 0 0 0 0 0 0 0 0 0 [12,] 0 0 0 0 0 0 0 0 0 [13,] 0 0 0 0 0 0 0 0 0
[14,] 0 0 0 0 0 0 0 0 0 [15,] 0 0 0 0 0 0 0 0 0 [16,] 0 0 0 0 0 0 0 0 0 [17,] 0 0 0 0 0 0 0 0 0 [18,] 0
0 0 0 0 0 0 0 0 [reached 'max' / getOption("max.print") – omitted 36 rows]

`TIMESTEP` [1] "0.0001"

`TOT` [1] "100"

`VOL` [1] "30.31"

$APPLY COMPENSATION [1] "FALSE"

$CHARSET [1] "utf-8"

$CREATOR [1] "SpectroFlo 3.3.0"

$FCSversion [1] "3"

$FILENAME [1] "C:/Users/Macarena/CytometryInR/course/03_InsideFCSFile/data/CellCounts4L_AB_05_ND050_05.fcs"

$FSC ASF [1] "1.21"

$GROUPNAME [1] "ND050"

$GUID [1] "CellCounts4L_AB_05-ND050-05.fcs"

Proj: Often corresponding to the experiment file name

```
DescriptionList$`$PROJ`
```

[1] "CellCounts4L_AB_05"

Spillover: Where the internal spillover matrix is stored (we will revisit during compensation)

```
DescriptionList$`$SPILLOVER`
```

UV1-A UV2-A UV3-A UV4-A UV5-A UV6-A UV7-A UV8-A UV9-A UV10-A UV11-A UV12-A UV13-A UV14-A UV15-A UV16-A V1-A V2-A V3-A V4-A V5-A [1,] 1e+00 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [2,] 1e-06 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [3,] 0e+00 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [4,] 0e+00 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [5,] 0e+00 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [6,] 0e+00 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [7,] 0e+00 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [8,] 0e+00 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [9,] 0e+00 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [10,] 0e+00 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [11,] 0e+00 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [12,] 0e+00 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [13,] 0e+00 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [14,] 0e+00 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 [15,] 0e+00 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 [16,] 0e+00 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 [17,] 0e+00 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 [18,] 0e+00 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0

(this list is longer, I just paste a fragment of it)

TOT: Total events (in this case my downsampled 100 cells)

```
DescriptionList$`$TOT`
```

[1] “100”

Volume: Volume amount acquired during acquisition.

```
DescriptionList$`$VOL`
```

[1] “30.31”

Software: Software used and version

```
DescriptionList$CREATOR
```

[1] “SpectroFlo 3.3.0”

```
Detectors <- DescriptionList[390:398]
Detectors
```

$FCSversion [1] “3”

$FILENAME [1] “C:/Users/Macarena/CytometryInR/course/03_InsideFCSFile/data/CellCounts4L_AB_05_ND050_05.fcs”

$FSC ASF [1] “1.21”

$GROUPNAME [1] “ND050”

$GUID [1] “CellCounts4L_AB_05-ND050-05.fcs”

FILENAME: Basically the full file.path to the .fcs file of interest.

```
DescriptionList$FILENAME
```

[1] “C:/Users/Macarena/CytometryInR/course/03_InsideFCSFile/data/CellCounts4L_AB_05_ND050_05.fcs

GROUPNAME: The Name assigned to the acquisition Group.

```
DescriptionList$GROUPNAME
```

[1] “ND050”

Laser Metadata

There is also a list of keywords related with the individual lasers as far as delays and area scaling factors for a particular day (also useful when plotted).

```
Detectors <- DescriptionList[399:410]
Detectors
```

$LASER1ASF [1] “1.09”

$LASER1DELAY [1] “-19.525”

$LASER1NAME [1] “Violet”

$LASER2ASF [1] “1.14”

$LASER2DELAY [1] “0”

$LASER2NAME [1] “Blue”

$LASER3ASF [1] “1.02”

$LASER3DELAY [1] “20.15”

$LASER3NAME [1] “Red”

$LASER4ASF [1] “0.92”

$LASER4DELAY [1] “40.725”

$LASER4NAME [1] “UV”

Display

Then, it is important to determine if a particular detector needs to be displayed as linear (in the case of time and scatter) or as log (for individual detectors).

```
Detectors <- DescriptionList[412:472]
Detectors
```

$P10DISPLAY [1] “LOG”

$P11DISPLAY [1] “LOG”

$P12DISPLAY [1] “LOG”

$P13DISPLAY [1] “LOG”

$P14DISPLAY [1] “LOG”

$P15DISPLAY [1] “LOG”

$P16DISPLAY [1] “LOG”

$P17DISPLAY [1] “LOG”

$P18DISPLAY [1] “LIN”

$P19DISPLAY [1] “LIN”

$P1DISPLAY [1] “LOG”

$P20DISPLAY [1] “LOG”

$P21DISPLAY [1] “LOG”

$P22DISPLAY [1] “LOG”

$P23DISPLAY [1] “LOG”

$P24DISPLAY [1] “LOG”

$P25DISPLAY [1] “LOG”

$P26DISPLAY [1] “LOG”

$P27DISPLAY [1] “LOG”

$P28DISPLAY [1] “LOG”

$P29DISPLAY [1] “LOG”

$P2DISPLAY [1] “LOG”

$P30DISPLAY [1] “LOG”

$P31DISPLAY [1] “LOG”

$P32DISPLAY [1] “LOG”

$P33DISPLAY [1] “LOG”

$P34DISPLAY [1] “LOG”

$P35DISPLAY [1] “LOG”

$P36DISPLAY [1] “LIN”

$P37DISPLAY [1] “LIN”

$P38DISPLAY [1] “LIN”

$P39DISPLAY [1] “LIN”

$P3DISPLAY [1] “LOG”

$P40DISPLAY [1] “LOG”

$P41DISPLAY [1] “LOG”

$P42DISPLAY [1] “LOG”

$P43DISPLAY [1] “LOG”

$P44DISPLAY [1] “LOG”

$P45DISPLAY [1] “LOG”

$P46DISPLAY [1] “LOG”

$P47DISPLAY [1] “LOG”

$P48DISPLAY [1] “LOG”

$P49DISPLAY [1] “LOG”

$P4DISPLAY [1] “LOG”

$P50DISPLAY [1] “LOG”

$P51DISPLAY [1] “LOG”

$P52DISPLAY [1] “LOG”

$P53DISPLAY [1] “LOG”

$P54DISPLAY [1] “LOG”

$P55DISPLAY [1] “LOG”

$P56DISPLAY [1] “LOG”

$P57DISPLAY [1] “LOG”

$P58DISPLAY [1] “LOG”

$P59DISPLAY [1] “LOG”

$P5DISPLAY [1] “LOG”

$P60DISPLAY [1] “LOG”

$P61DISPLAY [1] “LOG”

$P6DISPLAY [1] “LOG”

$P7DISPLAY [1] “LOG”

$P8DISPLAY [1] “LOG”

$P9DISPLAY [1] “LOG”

And a few final keywords with threshold, window scaling and other user selected settings.

```
Detectors <- DescriptionList[473:476]
Detectors
```

$THRESHOLD [1] “(FSC,50000)”

$TUBENAME [1] “05”

$USERSETTINGNAME [1] “DTR_CellCounts”

$WINDOW EXTENSION [1] “3”

flowCore parameters

Depending on the arguments selected during read.FCS(), we might also encounter additional keywords that are added in by flowCore. For example, we do not see these keywords when “transformation” is set to FALSE.

```
flowCoreCheck <- read.FCS(filename=firstfile,
 transformation = FALSE, truncate_max_range = FALSE)
flowCoreCheck
```

flowFrame object ‘CellCounts4L_AB_05-ND050-05.fcs’ with 100 cells and 61 observables: name desc range minRange maxRange $P1TimeNA2721400272139P2$ UV1-A NA 4194304 -111 4194303 $P3UV2 - ANA4194304 - 1114194303P4$ UV3-A NA 4194304 -111 4194303 $P5UV4 - ANA4194304 - 1114194303$.....P57 R4-A NA 4194304 -111 4194303 $P58R5 - ANA4194304 - 1114194303P59$ R6-A NA 4194304 -111 4194303 $P60R7 - ANA4194304 - 1114194303P61$ R8-A NA 4194304 -111 4194303 476 keywords are stored in the ‘description’ slot

```
NoChange <- keyword(flowCoreCheck)
Detectors <- NoChange [476:500]
Detectors
```

$WINDOW EXTENSION [1] “3”

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

```
NoChange <- keyword(flowCoreCheck)
Detectors <- NoChange [476:500]
Detectors
```

$WINDOW EXTENSION [1] “3”

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

$ NULL

By contrast, if we had set “transformation” to TRUE:

```
flowCoreCheck <- read.FCS(filename=firstfile,
 transformation = TRUE, truncate_max_range = FALSE)
flowCoreCheck
```

flowFrame object 'CellCounts4L_AB_05-ND050-05.fcs' with 100 cells and 61 observables: name desc range minRange maxRange $P1TimeNA2721400272139P2$ UV1-A NA 4194304 -111 4194303 $P3UV2 - ANA4194304 - 1114194303P4$ UV3-A NA 4194304 -111 4194303 $P5UV4 - ANA4194304 - 1114194303$.....P57 R4-A NA 4194304 -111 4194303 $P58R5 - ANA4194304 - 1114194303P59$ R6-A NA 4194304 -111 4194303 $P60R7 - ANA4194304 - 1114194303P61$ R8-A NA 4194304 -111 4194303 599 keywords are stored in the 'description' slot

```
YesChange <- keyword(flowCoreCheck)
Detectors <- YesChange [476:500]
Detectors
```

$WINDOW EXTENSION [1] "3"

$transformation [1] "applied"

`$flowCore/P1Rmax`[1] "272140"

`$flowCore/P1Rmin`[1] "0"

`$flowCore/P2Rmax`[1] "4194304"

`$flowCore/P2Rmin`[1] "-111"

`$flowCore/P3Rmax`[1] "4194304"

`$flowCore/P3Rmin`[1] "-111"

`$flowCore/P4Rmax`[1] "4194304"

`$flowCore/P4Rmin`[1] "-111"

`$flowCore/P5Rmax`[1] "4194304"

`$flowCore/P5Rmin`[1] "-111"

`$flowCore/P6Rmax`[1] "4194304"

`$flowCore/P6Rmin`[1] "-111"

`$flowCore/P7Rmax`[1] "4194304"

`$flowCore/P7Rmin`[1] "-111"

`$flowCore/P8Rmax`[1] "4194304"

`$flowCore$/P8Rmin'[1] "-26.3464946746826"

`$flowCore$/P9Rmax'[1] "4194304"

`$flowCore$/P9Rmin'[1] "-111"

`$flowCore$/P10Rmax'[1] "4194304"

`$flowCore$/P10Rmin'[1] "0"

`$flowCore$/P11Rmax'[1] "4194304"

`$flowCore$/P11Rmin'[1] "-111"

`$flowCore$/P12Rmax'[1] "4194304"

For some flow cytometry R packages, you will notice when opening their exported .fcs outputs in commercial software that these flowCore keywords have ended up integrated. It is likely somewhere in the package code the author forgot to add set transformation to FALSE, which is why we are seeing these flowCore keywords after the fact.